

DEPARTMENT OF COMPUTER & SOFTWARE ENGINEERING

College of E&ME, NUST, Rawalpindi

CS-272 ARTIFICIAL INTELLIGENCE

Lab Report # 01: Object Oriented Design Practices in Python

Class / Semester	Semester 7 - Section B
Instructor	Dr. Arslan Shaukat
Lab Engineer	Engr. Eman Fatima
Lab No.	01
Date of Submission	16-09-2026

Table of Contents

1. Objectives.....	
2. Software Name and Usage.....	
3. Introduction.....	
4. Task 1 - Palindrome Checker.....	
4.1 Task Statement.....	
4.2 Python Code.....	
4.3 Output Screenshot.....	
4.4 Result.....	
5. Task 2 - Armstrong Number Checker.....	
5.1 Task Statement.....	
5.2 Python Code.....	
5.3 Output Screenshot.....	
5.4 Result.....	
6. Task 3 - Random Verification Code Generator.....	
6.1 Task Statement.....	
6.2 Python Code.....	
6.3 Output Screenshot.....	
6.4 Result.....	
7. Task 4 - Vector Class.....	
7.1 Task Statement.....	
7.2 Python Code.....	
7.3 Output Screenshot.....	
7.4 Result.....	
8. Task 5 - Binary Search Tree - Node Search.....	
8.1 Task Statement.....	
8.2 Python Code.....	
8.3 Time Complexity.....	
8.4 Output Screenshot.....	
8.5 Result.....	
9. Conclusion.....	
10. References.....	

1. Objectives

- Gain familiarity with the Python IDLE environment and standard Python coding conventions.
- Understand and apply object-oriented design principles such as classes, objects, and modular programming in Python.
- Use object-oriented constructs to build hierarchical data structures and model real-world problems effectively.

2. Software Name and Usage

Software Used: Python IDLE (Python 3)

Python 3, along with the IDLE development environment, was used to write, edit, and execute every script for this lab. Programs were run from a terminal using the command 'python3 <filename>.py', which invokes the Python interpreter to execute the source file and print results directly to the console.

3. Introduction

This lab builds on core Python fundamentals - operators, strings, conditional statements, loops, and functions - and extends them into object-oriented design. Python's class mechanism groups related data (attributes) and behaviour (methods) into a single reusable unit, which makes it possible to model real-world entities such as a mathematical vector or the nodes of a tree in a natural way.

The five tasks in this lab progress from simple string and numeric processing (checking a sentence for the palindrome property, and testing whether a number satisfies the Armstrong-number property) to constrained random generation (producing a verification code that must contain a mix of character types), and finally to object-oriented modelling proper: a Vector class that encapsulates coordinate data and geometric operations, and a Binary Search Tree implemented with a self-referential Node class, which is then used to demonstrate an $O(\log n)$ -class search operation.

4. Task 1 - Palindrome Checker

4.1 Task Statement

Write a program that takes a sentence from the user and checks whether it is a palindrome (ignoring spaces, punctuation, and case). Example: "Was it a car or a cat I saw" -> Palindrome.

4.2 Python Code

```
def check(word : str) -> bool:

    special_chars_list = [
        '!', '"', '#', '$', '%', '&', "'", '(', ')', '*', '+', ',', '-', '.', ':', '/',
        ':', ';', '<', '=', '>', '?', '@', '[', '\\', ']', '^', '_', '`', '{',
        '|', '}', '~', ' '
    ]
    for character in special_chars_list:
        word = word.replace(character, "")
    word = word.lower()
    length = len(word)

    for i in range(length):
```

```

        if (length - i == i): break

        if(word[i] == word[length-i-1]): continue

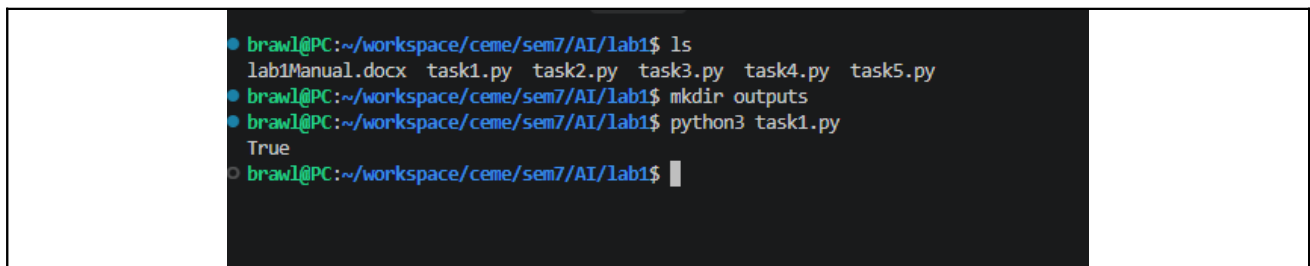
        return False
    return True

print(check("Was it a car or a cat I saw"))

```

The check() function first strips every punctuation/whitespace character and lower-cases the string, then compares characters from both ends inward to test the palindrome property. The example sentence from the lab manual is passed directly to the function; replacing the literal string with input() would let the sentence be typed in at runtime.

4.3 Output Screenshot



```

brawl@PC:~/workspace/ceme/sem7/AI/lab1$ ls
lab1Manual.docx task1.py task2.py task3.py task4.py task5.py
brawl@PC:~/workspace/ceme/sem7/AI/lab1$ mkdir outputs
brawl@PC:~/workspace/ceme/sem7/AI/lab1$ python3 task1.py
True
brawl@PC:~/workspace/ceme/sem7/AI/lab1$

```

Figure 1: Terminal output of the palindrome checker for the sentence "Was it a car or a cat I saw"

4.4 Result

Running the program on the sentence "Was it a car or a cat I saw" returned True, confirming that after removing spaces/punctuation and normalising case, the string reads the same forwards and backwards.

5. Task 2 - Armstrong Number Checker

5.1 Task Statement

Write a program that checks if a given number is an Armstrong number - a number that is equal to the sum of its own digits, each raised to the power of the number of digits.

5.2 Python Code

```

def check(num : int) -> bool :
    str_num = str(num)
    length = len(str_num)

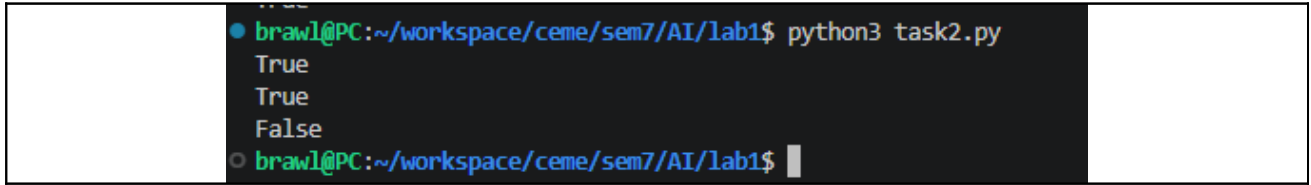
    sum = 0
    for digit in str_num :
        sum += int(digit) ** length

    return sum == num

print(check(1634))
print(check(153))
print(check(1234))

```

5.3 Output Screenshot



```

brawl@PC:~/workspace/ceme/sem7/AI/lab1$ python3 task2.py
True
True
False
brawl@PC:~/workspace/ceme/sem7/AI/lab1$

```

Figure 2: Output of the Armstrong number checker for 1634, 153, and 1234

5.4 Result

The function returned True for 1634 ($1^4 + 6^4 + 3^4 + 4^4 = 1634$) and True for 153 ($1^3 + 5^3 + 3^3 = 153$), correctly identifying both as Armstrong numbers, while 1234 returned False since it does not satisfy the property.

6. Task 3 - Random Verification Code Generator

6.1 Task Statement

Write a program that generates a random verification code consisting of letters, digits, and special characters. The user enters the desired length, and each generated code must include at least one uppercase letter, one digit, and one special character.

6.2 Python Code

```

import random

x = int(input("enter a number: "))

if x < 3 :
    print("minimum 3 characters required , generating a 3 character code")
    x = 3

code = ""

special_char_map = {
    1: '!',
    2: '@',
    3: '#',
    4: '$',
    5: '%'
}

lowercase_map = {
    1: 'a', 2: 'b', 3: 'c', 4: 'd', 5: 'e', 6: 'f', 7: 'g',
    8: 'h', 9: 'i', 10: 'j', 11: 'k', 12: 'l', 13: 'm', 14: 'n',
    15: 'o', 16: 'p', 17: 'q', 18: 'r', 19: 's', 20: 't', 21: 'u',
    22: 'v', 23: 'w', 24: 'x', 25: 'y', 26: 'z'
}

num_special = random.randint(1,int(x/3))
num_uppercase = random.randint(1 , int(x/3))
num_numerics = random.randint(1 , int(x/3))
x -= (num_numerics + num_special + num_uppercase)

```

```

for i in range(x):

    random_number = random.randint(1,26)
    code += lowercase_map[random_number]

for i in range(num_special):

    random_number = random.randint(1,5)
    code += special_char_map[random_number]

for i in range(num_numerics):

    random_number = random.randint(1,9)
    code += str(random_number)

for i in range(num_uppercase):

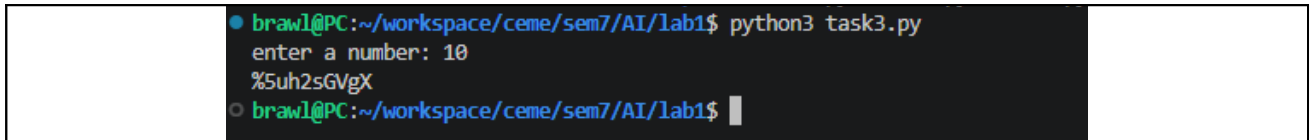
    random_number = random.randint(1,26)
    code += lowercase_map[random_number].upper()

code = "".join(random.sample(code, len(code)))
print(code)

```

Enforcing a minimum count of 1 for num_special, num_uppercase, and num_numerics guarantees that every generated code contains at least one special character, one digit, and one uppercase letter; the remaining characters are filled with lowercase letters and the whole string is shuffled with random.sample() before being printed.

6.3 Output Screenshot



```

● brawl@PC:~/workspace/ceme/sem7/AI/lab1$ python3 task3.py
  enter a number: 10
    %5uh2sGVgX
○ brawl@PC:~/workspace/ceme/sem7/AI/lab1$

```

Figure 3: Verification code generated for an input length of 10

6.4 Result

For an input length of 10, the program produced the code "%5uh2sGVgX", which contains a special character (%), digits (5, 2), uppercase letters (G, X), and lowercase letters, satisfying every constraint from the task statement.

7. Task 4 - Vector Class

7.1 Task Statement

Create a class Vector with attributes x and y to represent coordinates. Include methods magnitude() to compute the vector's length and direction() to calculate its angle with the x-axis. Take input for the vector in the main program, create an object of the class, and display its magnitude and direction.

7.2 Python Code

```
import math
```

```

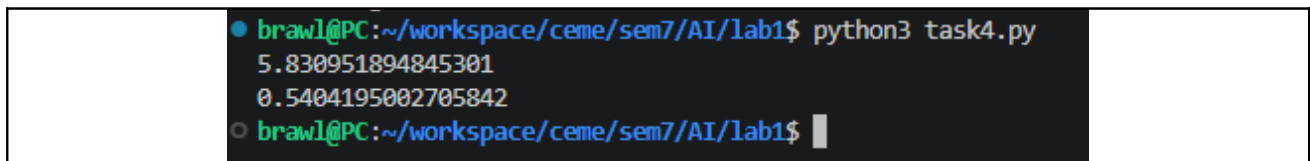
class MyVector:
    def __init__(self , x = 0.0 , y = 0.0) -> None:
        self.x = x
        self.y = y
        return
    def update_x(self , x:float) -> None:
        self.x = x
        return
    def update_y(self , y:float) -> None:
        self.y = y
        return
    def magnitude(self) -> float :
        return math.sqrt(self.x**2 + self.y**2)
    def direction(self) -> float :
        return math.atan2(self.y , self.x)

test_vector = MyVector(5.0 , 3.0)
print(test_vector.magnitude())
print(test_vector.direction())

```

The MyVector class encapsulates the x and y coordinates as attributes, with update_x()/update_y() to modify them, magnitude() computing $\sqrt{x^2 + y^2}$, and direction() using $\text{math.atan2}(y, x)$ to return the angle (in radians) with the x-axis.

7.3 Output Screenshot



```

braw1@PC:~/workspace/ceme/sem7/AI/lab1$ python3 task4.py
5.830951894845301
0.5404195002705842
braw1@PC:~/workspace/ceme/sem7/AI/lab1$

```

Figure 4: Magnitude and direction computed for the vector (5, 3)

7.4 Result

For the vector (5, 3), the program correctly computed a magnitude of 5.8309... units and a direction of 0.5404... radians, matching $\sqrt{5^2 + 3^2}$ and $\text{atan2}(3, 5)$ respectively.

8. Task 5 - Binary Search Tree - Node Search

8.1 Task Statement

Create a Binary Search Tree (BST) and insert a series of integers into it. Write a program to search for the node 25 in the BST, and state the time complexity of the search operation in Big-O notation.

8.2 Python Code

```

from typing import Any
class BSTNode:
    def exists(self, val: Any) -> bool:
        if val == self.val:
            return True

        if val < self.val:

```

```

        if self.left is None:
            return False
        return self.left.exists(val)

    if self.right is None:
        return False
    return self.right.exists(val)

def __init__(self, val: Any = None) -> None:
    self.left: "BSTNode | None" = None
    self.right: "BSTNode | None" = None
    self.val = val

def insert(self, val: Any) -> None:
    if not self.val:
        self.val = val
        return

    if self.val == val:
        return

    if val < self.val:
        if self.left:
            self.left.insert(val)
            return
        self.left = BSTNode(val)
        return

    if self.right:
        self.right.insert(val)
        return
    self.right = BSTNode(val)

tree = BSTNode()
tree.insert(15)
tree.insert(80)
tree.insert(30)
tree.insert(21)
tree.insert(25)

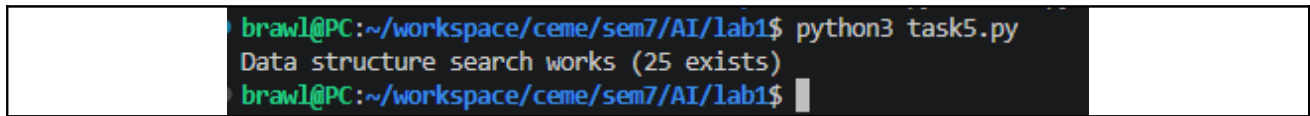
if(tree.exists(25)):
    print("Data structure search works (25 exists)")

```

8.3 Time Complexity

The `exists()` method walks down the tree once, moving to the left or right child based on a single comparison at each node, so it visits at most h nodes, where h is the height of the tree. This gives a time complexity of $O(h)$. For a balanced tree built from n randomly-ordered keys, h is approximately $\log_2(n)$, giving an average case of $O(\log n)$; in the worst case, where insertions produce a completely skewed (linked-list-like) tree, h approaches n and the search degrades to $O(n)$.

8.4 Output Screenshot

A terminal window with a dark background. The prompt is 'brawl@PC:~/workspace/ceme/sem7/AI/lab1\$'. The command 'python3 task5.py' has been executed, resulting in the output 'Data structure search works (25 exists)'. The prompt is now 'brawl@PC:~/workspace/ceme/sem7/AI/lab1\$' with a cursor at the end.

```
brawl@PC:~/workspace/ceme/sem7/AI/lab1$ python3 task5.py
Data structure search works (25 exists)
brawl@PC:~/workspace/ceme/sem7/AI/lab1$
```

Figure 5: Successful search for node 25 in the BST

8.5 Result

After inserting 15, 80, 30, 21, and 25 into the BST, `exists(25)` returned `True`, printing "Data structure search works (25 exists)", which confirms both the `insert()` and `exists()` methods operate correctly on the tree structure.

9. Conclusion

This lab provided hands-on practice with Python's core syntax as well as its object-oriented features. Working through the palindrome and Armstrong-number checks reinforced string manipulation and digit-level numeric processing, while the verification-code generator demonstrated how randomisation can be constrained to satisfy specific composition rules. Building the `MyVector` class showed how attributes and methods can be grouped to model a real-world geometric entity, and implementing the `BSTNode` class - with its recursive `insert()` and `exists()` methods - illustrated how a hierarchical, self-referential class can represent a binary search tree and how its structure bounds the cost of a search to $O(h)$. Overall, the lab met its objective of introducing object-oriented design practices and modular programming in Python.

10. References

1. CS-272 Artificial Intelligence, Lab Manual - 01: Object Oriented Design Practices in Python, Department of Computer & Software Engineering, College of E&ME, NUST, Rawalpindi.